

Tutorial: Cadastro de Clientes

com Streamlit e SQLite

Aplicacao CRUD completa com módulos separados

Versao 1.0 | 2026

Introdução

Este tutorial orienta a criação de uma aplicação web de cadastro de clientes utilizando Streamlit como framework de interface e SQLite como banco de dados. A aplicação será estruturada em dois módulos distintos, seguindo boas práticas de separação de responsabilidades.

A aplicação oferecerá as seguintes funcionalidades:

- CRUD completo de clientes (criar, consultar, atualizar e excluir)
- Cadastro e exclusão de cidades
- Relacionamento entre clientes e cidades via chave estrangeira
- Interface web interativa com Streamlit

Estrutura de módulos:

- database.py - Operações com o banco de dados SQLite
- app.py - Interface Streamlit para interação do usuário

Passo 1: Preparacao do Ambiente

1.1 Prerequisitos

Antes de iniciar, certifique-se de ter o Python 3.8 ou superior instalado. Você pode verificar a versão com o comando:

```
python --version
```

1.2 Criando o Ambiente Virtual

É uma boa pratica criar um ambiente virtual isolado para o projeto. Execute os seguintes comandos no terminal:

No Windows:

```
# Criar o ambiente virtual
python -m venv venv

# Ativar o ambiente virtual
venv\Scripts\activate
```

No Linux/macOS:

```
# Criar o ambiente virtual
python -m venv venv

# Ativar o ambiente virtual
source venv/bin/activate
```

1.3 Instalando as Dependências

Com o ambiente virtual ativado, instale as bibliotecas necessarias:

```
pip install streamlit
```

O SQLite ja esta incluído na biblioteca padrao do Python (módulo sqlite3), portanto não é necessária nenhuma instalação adicional para o banco de dados.

Para verificar se a instalação foi bem-sucedida, execute:

```
streamlit --version
```

1.4 Estrutura do Projeto

Crie uma pasta para o projeto e, dentro dela, os dois arquivos principais:

```
cadastro_clientes/
  database.py      # módulo de banco de dados
  app.py           # Interface Streamlit
```

Passo 2: módulo de Banco de Dados (database.py)

O módulo `database.py` centraliza todas as interações com o SQLite. Ele é responsável pela criação das tabelas, pela conexão com o banco e por todas as operações de dados.

2.1 Importações e Conexão

Inicie o arquivo importando o módulo `sqlite3` e criando a função de conexão:

```
import sqlite3

DB_NAME = "clientes.db"

def get_connection():
    """Retorna uma conexão com o banco de dados SQLite."""
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row # permite acesso por nome de coluna
    return conn
```

Nota: **`sqlite3.Row`** permite acessar os campos pelo nome da coluna (ex: `row["nome"]`) em vez de índice numérico, tornando o código mais legível.

2.2 Criação das Tabelas

Adicione a função que cria as tabelas `idades` e `clientes` no banco de dados:

```
def criar_tabelas():
    """Cria as tabelas cidades e clientes caso não existam."""
    conn = get_connection()
    cursor = conn.cursor()

    # Tabela de cidades
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS cidades (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL UNIQUE
        )
    """)

    # Tabela de clientes
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS clientes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            email TEXT NOT NULL UNIQUE,
            cidade_id INTEGER NOT NULL,
            FOREIGN KEY (cidade_id) REFERENCES cidades(id)
        )
    """)

    conn.commit()
    conn.close()
```

2.3 Operações com Cidades

Incluir Cidade

```
def incluir_cidade(nome: str) -> bool:
    """Insere uma nova cidade. Retorna True em caso de sucesso."""
    try:
        conn = get_connection()
        conn.execute(
            "INSERT INTO cidades (nome) VALUES (?)",
            (nome.strip(),)
        )
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False # cidade ja existe
    finally:
        conn.close()
```

Excluir Cidade

```
def excluir_cidade(cidade_id: int) -> bool:
    """
    Exclui uma cidade pelo ID.
    Retorna False se houver clientes vinculados.
    """
    conn = get_connection()
    try:
        # Verifica se ha clientes vinculados
        row = conn.execute(
            "SELECT COUNT(*) FROM clientes WHERE cidade_id = ?",
            (cidade_id,)
        ).fetchone()
        if row[0] > 0:
            return False

        conn.execute("DELETE FROM cidades WHERE id = ?", (cidade_id,))
        conn.commit()
        return True
    finally:
        conn.close()
```

Consultar Cidades

```
def listar_cidades() -> list:
    """Retorna todas as cidades ordenadas por nome."""
    conn = get_connection()
    try:
        rows = conn.execute(
            "SELECT id, nome FROM cidades ORDER BY nome"
        ).fetchall()
        return [dict(r) for r in rows]
    finally:
        conn.close()
```

2.4 Operações com Clientes (CRUD)

Incluir Cliente (Create)

```
def incluir_cliente(nome: str, email: str, cidade_id: int) -> bool:
    """Insere um novo cliente. Retorna True em caso de sucesso."""
    try:
        conn = get_connection()
        conn.execute(
            """INSERT INTO clientes (nome, email, cidade_id)
            VALUES (?, ?, ?)""",
            (nome.strip(), email.strip(), cidade_id)
        )
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False # email duplicado
    finally:
        conn.close()
```

Consultar Clientes (Read)

```
def listar_clientes() -> list:
    """
    Retorna todos os clientes com o nome da cidade (JOIN).
    """
    conn = get_connection()
    try:
        rows = conn.execute("""
            SELECT c.id, c.nome, c.email,
                   ci.nome AS cidade, c.cidade_id
            FROM    clientes c
            JOIN    cidades ci ON ci.id = c.cidade_id
            ORDER  BY c.nome
            """).fetchall()
        return [dict(r) for r in rows]
    finally:
        conn.close()

def buscar_cliente(cliente_id: int) -> dict | None:
    """Retorna os dados de um cliente pelo ID."""
    conn = get_connection()
    try:
        row = conn.execute(
            "SELECT id, nome, email, cidade_id FROM clientes WHERE id = ?",
            (cliente_id,)
        ).fetchone()
        return dict(row) if row else None
    finally:
        conn.close()
```

Alterar Cliente (Update)

```
def alterar_cliente(
```

```

    cliente_id: int,
    nome: str,
    email: str,
    cidade_id: int
) -> bool:
    """Atualiza os dados de um cliente. Retorna True em caso de sucesso."""
    try:
        conn = get_connection()
        conn.execute(
            """UPDATE clientes
               SET nome = ?, email = ?, cidade_id = ?
               WHERE id = ?""",
            (nome.strip(), email.strip(), cidade_id, cliente_id)
        )
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False # email duplicado em outro registro
    finally:
        conn.close()

```

Excluir Cliente (Delete)

```

def excluir_cliente(cliente_id: int) -> bool:
    """Remove um cliente pelo ID. Retorna True em caso de sucesso."""
    conn = get_connection()
    try:
        conn.execute("DELETE FROM clientes WHERE id = ?", (cliente_id,))
        conn.commit()
        return True
    finally:
        conn.close()

```

2.5 Código Completo do database.py

Abaixo o código completo e consolidado do módulo database.py:

```

# database.py
import sqlite3

DB_NAME = "clientes.db"

def get_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn

def criar_tabelas():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS cidades (
            id    INTEGER PRIMARY KEY AUTOINCREMENT,
            nome  TEXT NOT NULL UNIQUE
        )
    """)

```

```

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS clientes (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                nome          TEXT      NOT NULL,
                email         TEXT      NOT NULL UNIQUE,
                cidade_id     INTEGER NOT NULL,
                FOREIGN KEY (cidade_id) REFERENCES cidades(id)
            )
        """)
    conn.commit()
    conn.close()

# --- CIDADES ---

def incluir_cidade(nome: str) -> bool:
    try:
        conn = get_connection()
        conn.execute("INSERT INTO cidades (nome) VALUES (?)", (nome.strip(),))
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False
    finally:
        conn.close()

def excluir_cidade(cidade_id: int) -> bool:
    conn = get_connection()
    try:
        row = conn.execute(
            "SELECT COUNT(*) FROM clientes WHERE cidade_id = ?", (cidade_id,)
        ).fetchone()
        if row[0] > 0:
            return False
        conn.execute("DELETE FROM cidades WHERE id = ?", (cidade_id,))
        conn.commit()
        return True
    finally:
        conn.close()

def listar_cidades() -> list:
    conn = get_connection()
    try:
        rows = conn.execute("SELECT id, nome FROM cidades ORDER BY
nome").fetchall()
        return [dict(r) for r in rows]
    finally:
        conn.close()

# --- CLIENTES ---

def incluir_cliente(nome: str, email: str, cidade_id: int) -> bool:
    try:
        conn = get_connection()
        conn.execute(
            "INSERT INTO clientes (nome, email, cidade_id) VALUES (?, ?, ?)",
            (nome.strip(), email.strip(), cidade_id),
        )
    
```



```

        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False
    finally:
        conn.close()

def listar_clientes() -> list:
    conn = get_connection()
    try:
        rows = conn.execute("""
            SELECT c.id, c.nome, c.email, ci.nome AS cidade, c.cidade_id
            FROM   clientes c
            JOIN   cidades ci ON ci.id = c.cidade_id
            ORDER BY c.nome
            """).fetchall()
        return [dict(r) for r in rows]
    finally:
        conn.close()

def buscar_cliente(cliente_id: int) -> dict | None:
    conn = get_connection()
    try:
        row = conn.execute(
            "SELECT id, nome, email, cidade_id FROM clientes WHERE id = ?",
            (cliente_id, ),
        ).fetchone()
        return dict(row) if row else None
    finally:
        conn.close()

def alterar_cliente(cliente_id: int, nome: str, email: str, cidade_id: int) -> bool:
    try:
        conn = get_connection()
        conn.execute(
            "UPDATE clientes SET nome=?, email=?, cidade_id=? WHERE id=?",
            (nome.strip(), email.strip(), cidade_id, cliente_id),
        )
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False
    finally:
        conn.close()

def excluir_cliente(cliente_id: int) -> bool:
    conn = get_connection()
    try:
        conn.execute("DELETE FROM clientes WHERE id = ?", (cliente_id,))
        conn.commit()
        return True
    finally:
        conn.close()

```

Passo 3: módulo de Interface Streamlit (app.py)

O módulo app.py contém toda a interface gráfica da aplicação. Ele importa as funções do database.py e utiliza os componentes do Streamlit para exibir formulários, tabelas e mensagens ao usuário.

3.1 Estrutura Geral do app.py

O arquivo é organizado em seções utilizando st.sidebar para navegação:

```
# app.py
import streamlit as st
import database as db

# Garante que as tabelas existam ao iniciar a aplicacao
db.criar_tabelas()

st.title("Cadastro de Clientes")

menu = st.sidebar.selectbox(
    "Menu",
    ["Clientes", "Cidades"]
)

if menu == "Clientes":
    pagina_clientes()
else:
    pagina_cidades()
```

3.2 Pagina de Cidades

A pagina de cidades oferece inclusão e exclusão. Adicione a função abaixo:

```
def pagina_cidades():
    st.header("Gestao de Cidades")

    # --- Incluir Cidade ---
    st.subheader("Nova Cidade")
    with st.form("form_cidade", clear_on_submit=True):
        nome_cidade = st.text_input("Nome da Cidade")
        submitted = st.form_submit_button("Adicionar")
        if submitted:
            if not nome_cidade.strip():
                st.warning("Informe o nome da cidade.")
            elif db.incluir_cidade(nome_cidade):
                st.success(f"Cidade '{nome_cidade}' adicionada com sucesso!")
            else:
                st.error("Ja existe uma cidade com esse nome.")

    # --- Listar e Excluir Cidades ---
    st.subheader("Cidades Cadastradas")
    cidades = db.listar_cidades()
    if not cidades:
        st.info("Nenhuma cidade cadastrada.")
```

```

else:
    for cidade in cidades:
        col1, col2 = st.columns([4, 1])
        col1.write(cidade["nome"])
        if col2.button("Excluir", key=f"del_cidade_{cidade['id']}" ):
            if db.excluir_cidade(cidade["id"]):
                st.success("Cidade excluida.")
                st.rerun()
            else:
                st.error(
                    "Nao e possivel excluir: ha clientes vinculados a esta
cidade."
                )

```

3.3 Pagina de Clientes

A pagina de clientes oferece o CRUD completo. E dividida em três abas: listar, cadastrar/editar e excluir.

```

def pagina_clientes():
    st.header("Gestao de Clientes")

    aba_lista, aba_form, aba_excluir = st.tabs(
        ["Lista de Clientes", "Cadastrar / Editar", "Excluir"]
    )

    cidades = db.listar_cidades()
    opcoes_cidade = {c["nome"]: c["id"] for c in cidades}

    # ---- ABA: Lista ----
    with aba_lista:
        clientes = db.listar_clientes()
        if not clientes:
            st.info("Nenhum cliente cadastrado.")
        else:
            st.dataframe(
                [{
                    "ID": c["id"],
                    "Nome": c["nome"],
                    "E-mail": c["email"],
                    "Cidade": c["cidade"],
                } for c in clientes],
                use_container_width=True,
            )

    # ---- ABA: Cadastrar / Editar ----
    with aba_form:
        if not cidades:
            st.warning("Cadastre ao menos uma cidade antes de incluir
clientes.")
            return

        st.write("Para editar, informe o ID do cliente. Deixe em branco para
cadastrar novo.")
        id_editor = st.number_input(
            "ID do Cliente (para edicao)", min_value=0, step=1, value=0

```

```

    )

    # Pre-preenche se ID valido
    dados = db.buscar_cliente(int(id_editar)) if id_editar else None

    with st.form("form_cliente", clear_on_submit=True):
        nome = st.text_input(
            "Nome", value=dados["nome"] if dados else ""
        )
        email = st.text_input(
            "E-mail", value=dados["email"] if dados else ""
        )
        nomes_cidades = list(opcoes_cidade.keys())
        idx_default = 0
        if dados:
            cid_atual = next(
                (n for n, i in opcoes_cidade.items() if i ==
dados["cidade_id"]),
                None,
            )
            if cid_atual in nomes_cidades:
                idx_default = nomes_cidades.index(cid_atual)
        cidade_sel = st.selectbox(
            "Cidade", nomes_cidades, index=idx_default
        )
        submitted = st.form_submit_button(
            "Salvar" if not dados else "Atualizar"
        )

        if submitted:
            if not nome.strip() or not email.strip():
                st.warning("Preencha todos os campos.")
            else:
                cid = opcoes_cidade[cidade_sel]
                if dados:
                    ok = db.alterar_cliente(int(id_editar), nome, email,
cid)

                    msg_ok = "Cliente atualizado com sucesso!"
                else:
                    ok = db.incluir_cliente(nome, email, cid)
                    msg_ok = "Cliente cadastrado com sucesso!"
                if ok:
                    st.success(msg_ok)
                else:
                    st.error("Erro: e-mail ja cadastrado para outro
cliente.")

    # ---- ABA: Excluir ----
    with aba_excluir:
        clientes = db.listar_clientes()
        if not clientes:
            st.info("Nenhum cliente para excluir.")
        else:
            opcoes_cliente = {
                f"{c['id']} - {c['nome']}": c["id"] for c in clientes
            }
            sel = st.selectbox("Selecione o cliente",
list(opcoes_cliente.keys()))

```

```
if st.button("Excluir Cliente", type="primary"):
    db.excluir_cliente(opcoes_cliente[sel])
    st.success("Cliente excluido com sucesso!")
    st.rerun()
```

3.4 Código Completo do app.py

Abaixo o código completo e consolidado do módulo app.py:

```
# app.py
import streamlit as st
import database as db

db.criar_tabelas()

st.set_page_config(page_title="Cadastro de Clientes", layout="wide")
st.title("Cadastro de Clientes")

def pagina_cidades():
    st.header("Gestao de Cidades")
    st.subheader("Nova Cidade")
    with st.form("form_cidade", clear_on_submit=True):
        nome_cidade = st.text_input("Nome da Cidade")
        if st.form_submit_button("Adicionar"):
            if not nome_cidade.strip():
                st.warning("Informe o nome da cidade.")
            elif db.incluir_cidade(nome_cidade):
                st.success(f"Cidade '{nome_cidade}' adicionada!")
            else:
                st.error("Ja existe uma cidade com esse nome.")
    st.subheader("Cidades Cadastradas")
    cidades = db.listar_cidades()
    if not cidades:
        st.info("Nenhuma cidade cadastrada.")
    else:
        for cidade in cidades:
            col1, col2 = st.columns([4, 1])
            col1.write(cidade["nome"])
            if col2.button("Excluir", key=f"del_{cidade['id']}"):
                if db.excluir_cidade(cidade["id"]):
                    st.success("Cidade excluida.")
                    st.rerun()
                else:
                    st.error("Ha clientes vinculados a esta cidade.")

def pagina_clientes():
    st.header("Gestao de Clientes")
    cidades = db.listar_cidades()
    opcoes_cidade = {c["nome"]: c["id"] for c in cidades}
    aba_lista, aba_form, aba_excluir = st.tabs(
        ["Lista de Clientes", "Cadastrar / Editar", "Excluir"]
    )
    with aba_lista:
        clientes = db.listar_clientes()
        if not clientes:
            st.info("Nenhum cliente cadastrado.")
```

```

else:
    st.dataframe(
        [{"ID": c["id"], "Nome": c["nome"],
          "E-mail": c["email"], "Cidade": c["cidade"]}
         for c in clientes],
        use_container_width=True,
    )
with aba_form:
    if not cidades:
        st.warning("Cadastre ao menos uma cidade.")
        return
    id_editar = st.number_input("ID para edicao (0 = novo)", min_value=0,
step=1)
    dados = db.buscar_cliente(int(id_editar)) if id_editar else None
    with st.form("form_cliente", clear_on_submit=True):
        nome = st.text_input("Nome", value=dados["nome"] if dados else
"" )
        email = st.text_input("E-mail", value=dados["email"] if dados else
"" )
        nomes = list(opcoes_cidade.keys())
        idx = 0
        if dados:
            atual = next(
                (n for n, i in opcoes_cidade.items() if i ==
dados["cidade_id"]),
                None)
            if atual in nomes:
                idx = nomes.index(atual)
        cidade_sel = st.selectbox("Cidade", nomes, index=idx)
        if st.form_submit_button("Atualizar" if dados else "Salvar"):
            if not nome.strip() or not email.strip():
                st.warning("Preencha todos os campos.")
            else:
                cid = opcoes_cidade[cidade_sel]
                ok = (db.alterar_cliente(int(id_editar), nome, email, cid)
                     if dados else db.incluir_cliente(nome, email, cid))
                if ok:
                    st.success("Salvo com sucesso!")
                else:
                    st.error("E-mail ja utilizado por outro cliente.")
    with aba_excluir:
        clientes = db.listar_clientes()
        if not clientes:
            st.info("Nenhum cliente para excluir.")
        else:
            opcoes = {f"{c['id']} - {c['nome']}": c["id"] for c in clientes}
            sel = st.selectbox("Selecione o cliente", list(opcoes.keys()))
            if st.button("Excluir Cliente", type="primary"):
                db.excluir_cliente(opcoes[sel])
                st.success("Excluido com sucesso!")
                st.rerun()

menu = st.sidebar.selectbox("Menu", ["Clientes", "Cidades"])
if menu == "Clientes":
    pagina_clientes()
else:
    pagina_cidades()

```


Executando a Aplicação

Com os dois arquivos criados, execute a aplicação com o comando:

```
streamlit run app.py
```

O Streamlit abrirá automaticamente o navegador com a aplicação rodando em <http://localhost:8501>.

Sugestão de Uso

1. Acesse o menu Cidades e cadastre algumas cidades primeiro.
2. Depois acesse o menu Clientes para cadastrar, editar e excluir clientes.
3. Para editar um cliente, anote o ID listado na aba 'Lista de Clientes' e informe na aba 'Cadastrar / Editar'.

Observações Importantes

- O banco de dados clientes.db será criado automaticamente na pasta do projeto na primeira execução.
- Para reiniciar o banco do zero, basta excluir o arquivo clientes.db.
- A aplicação valida e-mails duplicados e impede a exclusão de cidades que possuam clientes vinculados.

--- Fim do Tutorial ---